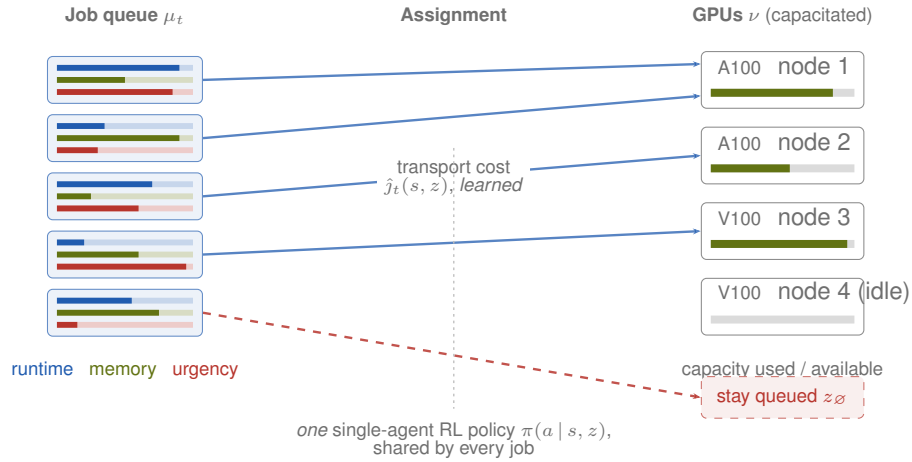


Project type: MA / SA
Project title: Optimal Transport for Learning to Schedule GPU Clusters

Student:
Supervisor: Lucia Pezzetti (lpezzetti@ethz.ch)
Supervising professor:

Semester: Spring 2027
Start date - end date:
Office space:



SALT applied to a cluster: a capacity-constrained optimal transport problem decides which job goes where, using a learned single-agent cost-to-go $\hat{j}_t(s, z)$ as its transport cost. Jobs that do not fit are transported to a “stay queued” target. One shared policy then controls every job.

Project description. In 2024 data centers used about $415TWh$ of electricity. Scheduling jobs on a shared GPU cluster is a coordination problem in which many jobs compete for a few heterogeneous accelerators, each job has its own runtime, memory footprint and urgency, and nobody knows in advance how long a job will actually take. Thus, deciding *which job goes where* and *how each job should then be run* are tightly coupled decisions, and solving them jointly quickly becomes intractable as the queue grows.

In recent work in our group we proved a *separation principle* for exactly this kind of structure and turned it into a family of algorithms called SALT (Separation-based Assignment and Learning via optimal Transport) [4, 6]. The idea: cast the population of agents as a probability measure, and the desired configuration as a target measure. The population cost-to-go is then upper bounded by an *optimal transport* problem whose transport cost is the *single-agent* cost-to-go $\hat{j}_t(s, z)$ of reaching target z from state s . This means you only ever learn a single-agent, target-conditioned policy — cheap, and independent of how many agents there are — and then coordinate the fleet by solving a transport problem. On a

Manhattan-scale vehicle routing task, SALT trained with 20 vehicles transfers without retraining to fleets of 1–100 and beats standard MARL baselines, which do not even run at that scale.

This project takes SALT out of its original mobility setting and asks whether the same decomposition works for computing infrastructure. The two ingredients that make it interesting here, and that are *not* covered by the current theory, are:

- **Capacity constraints.** In SALT the target measure ν is given: you prescribe exactly how much mass each target receives. In a cluster you do not want to prescribe the load of each GPU — you want capacity *upper bounds* and let the assignment decide the rest. This calls for capacitated (semi-relaxed) rather than balanced optimal transport.
- **Oversubscription.** When there are more jobs than the cluster can hold, no feasible assignment exists at all and some jobs must stay in the queue. This is where an unbalanced formulation [1] genuinely becomes necessary — although, as a first step, it can be handled exactly by adding a “stay queued” target that absorbs the excess mass at a deferral cost (see the figure above).

Motivation. Cluster schedulers today are mostly hand-tuned heuristics (shortest-job-first, backfilling, Tetris), and the learning-based alternatives [2, 3, 5] train monolithic policies that must be re-trained whenever the cluster or the workload changes. SALT offers a different bargain: learn one small, transferable model of *how a single job behaves*, and let a transport problem handle the combinatorics. Concretely, we want to understand whether the separation principle survives capacity constraints on the assignment, how large the separation gap actually is when job runtimes are highly uncertain, and whether one can train on a short queue and schedule a long one without retraining.

Approach. The mapping is a direct analogue of the vehicle-routing instance of SALT. A job is an agent; its state s collects remaining work, memory currently held, current placement, deadline slack and urgency; the target z is the GPU it is assigned to. The single-agent problem is not trivial: getting a job onto its assigned GPU costs queueing, cold start, data or checkpoint staging and possibly migration, all under stochastic contention — this is the “routing” component, and it is what $\hat{j}_t(s, z)$ has to learn. Exactly as “current position” and “destination” are distinct in the mobility setting, “current placement” lives in the state and “assigned placement” is the target. On top of this, you will replace the balanced transport problem by a capacitated one and check whether the separation bound still holds, verify that an optimal capacitated coupling can still be taken to place each job wholly on one GPU (which is what makes the bound deployable), and build a simulator to evaluate against classical and learned schedulers.

Goals.

1. Formulate cluster scheduling as a SALT instance: single-agent job dynamics, per-job additive costs, capacitated transport at the coordination layer.
2. Extend the separation principle to capacity-constrained transport, and establish the corresponding integrality (“one job, one GPU”) result.
3. Build a job-level simulator, implement SALT with a min-cost-flow coordination layer, and *measure the separation gap empirically* against an exactly solvable small instance.
4. Evaluate scalability and zero-shot transfer across queue lengths, against heuristic (SJF, backfilling, Tetris) and MARL baselines.
5. (Optional) Real workload traces [7], online job arrivals via a receding horizon, or a genuinely unbalanced transport formulation.

Scope, and what we already know is hard. We would rather be upfront about the boundaries of the framework than have you discover them in month four:

- Capacity must enter as a constraint on the *transport plan*, not on the set of feasible actions. Memory contention is therefore modelled as a *slowdown* (which the theory covers, with a quantified penalty), not as a hard mask that forbids a job from starting.

- The objective has to be additive across jobs plus a terminal transport cost. Weighted completion time, tardiness, mean slowdown and energy are fine. Makespan and max–min fairness are not expressible and stay out of scope (we may still report them as diagnostics).
- The main results use a fixed set of queued jobs per episode; online arrivals are an extension, handled by re-solving over the currently queued jobs each epoch. Heterogeneity across jobs lives in the *state*, so that a single shared policy remains valid.

The separation bound degrades with the dispersion of the dynamics, and real GPU job runtimes are heavy-tailed — so measuring the gap numerically (Goal 3) is a deliberate, and interesting, part of the project rather than an afterthought.

Qualifications. We are searching for a highly motivated student with:

- Strong analytical skills; solid Python (RL libraries and JAX/PyTorch welcome).
- Background in control, optimization or reinforcement learning.
- Familiarity with MDPs is expected; exposure to optimal transport, network flows or systems/scheduling is a plus but can be picked up along the way.

Depending on your interests, the project can lean towards the theory (Goal 2) or towards the implementation and empirical study (Goals 3–4). A semester project would focus on the latter.

How to apply. Please send your CV and transcript to lpezetti@ethz.ch, together with a couple of sentences on which part of the project appeals to you. Happy to discuss informally before you decide.

Tasks list.

1. **Literature review.** SALT and dynamic programming in probability spaces; capacitated and unbalanced optimal transport; cluster scheduling heuristics and learned schedulers. Fix assumptions and evaluation metrics.
2. **Modeling.** Specify the single-agent job MDP (states, actions, stochastic throughput and staging), the per-job cost structure, and the capacitated transport problem with a deferral target.
3. **Theory.** Re-derive the separation bound with the transport plan restricted by capacity constraints; prove the integrality result that makes the resulting assignment deployable.
4. **Simulation environment.** Build a job-level cluster simulator with heterogeneous GPUs, stochastic runtimes and configurable oversubscription; add an exactly solvable small regime for validation.
5. **Algorithms.** Implement SALT with a min-cost-flow coordination layer; single-agent learner of your choice (Q-learning / PPO).
6. **Evaluation.** Measure the separation gap against exact dynamic programming; compare against heuristic and MARL baselines; study transfer across queue lengths.
7. **Reporting.** Final report, presentation, and a manuscript outline if the results support it.

Timeline.

- **Month 1:** T1–T2. Milestone: the single-agent job problem is written down and non-trivial.
- **Month 2:** T2–T3. Milestone: the capacitated bound holds, or we fall back to the deferral-target reduction and reweight towards experiments.
- **Month 3:** T4 (simulator) and T5 (algorithms).
- **Month 4:** T6, starting with the separation-gap measurement on small instances.
- **Months 5–6:** T6 (baselines, scalability and transfer) and T7 (writing).

References

- [1] Lenaïc Chizat, Gabriel Peyré, Bernhard Schmitzer, and François-Xavier Vialard. Scaling algorithms for unbalanced optimal transport problems. *Mathematics of Computation*, 87(314):2563–2609, 2018.
- [2] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, pages 50–56, 2016.
- [3] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM SIGCOMM Conference*, pages 270–288, 2019.
- [4] Lucia Pezzetti, Nicolas Lanzetti, Antonio Terpin, Florian Dörfler, and Giorgia Ramponi. A separation principle for cooperative multi-agent reinforcement learning, 2026. Under review, NeurIPS 2026.
- [5] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 1–18, 2021.
- [6] Antonio Terpin, Nicolas Lanzetti, and Florian Dörfler. Dynamic programming in probability spaces via optimal transport. *SIAM Journal on Control and Optimization*, 62(2):1183–1206, 2024.
- [7] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 945–960, 2022.